

Enhancing Fashion-MNIST Classification Using a Hybrid CNN–Vision Transformer Approach

Abstract

This project looks at how combining Convolutional Neural Networks (CNNs) with Vision Transformers (ViTs) can make image classification work better on the Fashion-MNIST dataset. I started with a plain Vision Transformer model that reached 83.80 % accuracy and then made three key changes. First, I added a few convolutional layers before the transformer so the model could pick up basic image patterns early on. Second, I applied stronger data-augmentation techniques like MixUp and small image rotations to give the model more variety to learn from. Third, I tuned the training process itself by using label smoothing and a cosine-annealing learning-rate schedule with a short warm-up at the beginning.

After testing these changes, the final model reached 90.22 % accuracy, roughly 6.4 percentage points higher than the baseline, and trained in about 143 minutes instead of 326. The outcome demonstrates that a hybrid CNN–Transformer model can work extremely well even on small grayscale datasets such as Fashion-MNIST. It's also a good reminder that a few well-planned tweaks can have a significant impact on model performance without simply increasing model size.

Introduction

Online fashion platforms today deal with thousands of new products every month. Keeping up with this growth requires automated systems that can quickly recognise and categorise images of clothing items. Manually having to label images is slow, subjective, and simply not scalable. The Fashion-MNIST dataset provides a good benchmark for testing how well machine learning models can classify clothing images. It consists of 60,000 training and 10,000 test images, all grayscale and 28×28 pixels, divided evenly into ten clothing categories, ranging from t-shirts and trousers to sneakers and bags.

Traditional image classification tasks have been led by CNNs for a long time, which are excellent at learning local spatial features through their hierarchical convolutional layers. However, Vision Transformers trained from scratch can be more challenging to apply effectively on smaller datasets, making Fashion-MNIST a useful test case for investigating whether combining the two architectures, CNNs and ViTs can combine complementary strengths.

This project was designed with that goal in mind. By introducing convolutional preprocessing before the transformer layers and improving both data diversity and the training process, I aimed to build a more accurate and efficient image classifier.

Objectives

For this project, I wanted to:

1. Build a working baseline Vision Transformer and see how well it performs on Fashion-MNIST
2. Make 3 substantial modifications to improve the model's accuracy
3. Compare my baseline and modified versions thoroughly

4. Investigate the overall impact of the modifications and consider why they may have improved the model's performance

Project Scope

I'm following Option 2 for this project, which means I can use existing ViT code as a starting point but need to make significant modifications myself. All my changes are documented with comments in the code so it's clear what I added versus what was already there.

Background and Related Work

Vision Transformers take the same attention mechanism that made transformer models successful in natural-language tasks and apply it to vision. Instead of scanning an image pixel by pixel, a ViT divides it into small patches (each treated like a “word”) and learns how different patches relate to one another through self-attention. This global relationship modelling is what gives transformers an advantage over CNNs on large datasets.

CNNs, by contrast, learn features hierarchically: early layers capture edges, middle layers form shapes, and deeper ones understand objects. Their strength lies in local spatial awareness, although capturing bigger relationships can require deeper layers. Combining CNNs and transformers has therefore become a popular research topic, as each of them fills the other's gaps.

The project I based my baseline on was described by Abd Alaziz et al. (2023), who achieved 95.25 percent accuracy on Fashion-MNIST using a pure ViT. Their setup, though, was more computationally demanding and did not explore hybrid architectures. I wanted to investigate how well a hybrid approach could perform within the computational resources available for this project.

Another relevant idea comes from recent work on data augmentation. MixUp, introduced by Zhang et al. (2017), blends pairs of images and their labels to force smoother decision boundaries. Simple geometric transformations like small rotations and flips can also prevent overfitting by exposing the model to slight variations of each item. Similarly, label smoothing, introduced in the Inception architecture (Szegedy et al., 2016), reduces overconfidence by softening targets. Cosine-annealing schedules for the learning gradually adjust the learning rate during training.

All these methods inspired my own three-part improvement plan. Together, they provided a structured way to investigate the overall impact of architectural, data augmentation, and training modifications on final model performance.

Methodology Overview

My approach was to start with a baseline Vision Transformer implementation and then develop a modified version incorporating three main changes. Each modification targeted one area: model architecture, data preparation, and training procedure. The aim was to investigate the overall effect of these changes in the final system.

In total, I trained and evaluated two main models:

- a **baseline ViT**, serving as the control experiment, and
- a **hybrid CNN-ViT**, which included all three improvements.

Both models used the same dataset split and evaluation metrics for a fair comparison.

Baseline Vision Transformer

For the baseline, I used the standard ViT structure with small patches and moderate hidden dimensions, images were resized from 28x28 to 32x32 so they could be evenly divided into 4x4 patches, resulting in 64 patches per image. Each patch was flattened and projected into a 256-dimensional embedding vector, followed by positional encoding.

The transformer contained six encoder blocks, each with eight attention heads and a feed-forward dimension of 512. In total, the network had approximately 3.19 million parameters. Training used the Adam optimizer (learning rate = 0.001), standard cross-entropy loss, batch size = 128, and 10 epochs. Pixel values were normalized between -1 and 1.

This baseline achieved 83.80 % accuracy on the test set after 10 epochs. The training loss generally decreased over the course of training, although test performance varied between epochs. It set a good reference point for the experiments ahead.

Modification 1: Hybrid CNN–ViT Architecture

One of the biggest limitations of the plain Vision Transformer is that it doesn't really "see" small local features early on, it just chops the image into patches and immediately starts treating them as separate tokens. CNNs, on the other hand, are really good at catching things like edges, corners, and textures before thinking about the whole picture. So my first idea was to add a small convolutional network in front of the transformer to extract these low-level features first.

I built a simple two-layer CNN as a preprocessing block. The first layer had one input channel (since Fashion-MNIST images are grayscale) and 32 output channels, followed by batch normalization and ReLU activation. The second convolution increased the channels from 32 to 64, again followed by batch normalization and ReLU. After that, I added a max-pooling layer to reduce the feature map size from 32x32 to 16x16.

These processed feature maps were then passed to the Vision Transformer, which then divided the 16x16 feature maps into patches and processed them as tokens, instead of applying the patch embedding directly to the original one-channel image. This way, the ViT worked on a richer representation from the start.

The main reasoning was simple: let the CNN take care of fine visual details so the transformer can focus on bigger relationships. This provided a way to combine the local feature extraction of CNNs with the global attention mechanism of the transformer.

The modified model showed more consistent improvement during the early stages of training than the baseline, although the individual effect of the CNN cannot be isolated because the other modifications were introduced at the same time.

Modification 2: Advanced Data Augmentation

The next thing to tackle was the data itself. Fashion-MNIST isn't a huge dataset - 60,000 training images can feel tiny for a transformer model. So I looked into ways to artificially expand it through augmentation. I used a mix of MixUp and basic geometric transformations.

MixUp is a simple but powerful idea, it takes two random images and merges them together and the same thing goes for their labels. For example, if one image is a shirt and another is a coat, the model might see something that looks halfway between them, maybe around 60% shirt and 40% coat. The label is mixed in the same ratio, so the model learns to see that not everything belongs neatly to one category.

For my setup, I used a parameter called lambda. This was sampled from a Beta distribution with $\alpha = 0.2$. That value fit quite well, the images weren't completely blended, but they weren't too distinct either. It gave enough variation for the model to learn from without throwing it off.

Along with MixUp, I added a few geometric transformations to make the training data more diverse; things like small rotations, translations, and flips. Those extra variations were intended to help the model handle clothing items that are at slightly different angles or positions.

What's good about these augmentations is that they did more than just inflate the dataset. The final model showed more consistent improvement during training than the baseline, although the specific contribution of the augmentation techniques cannot be isolated from the original experiment.

Modification 3: Improved Training Strategy

The last modification was about training stability and optimisation. Transformers can be tricky to train, especially at the beginning. I implemented two techniques: **label smoothing** and **cosine annealing with warmup**.

Label Smoothing

Normally, a classification model is told that one class is "100% correct" and all others are "0%." Label smoothing slightly relaxes that rule. For example, instead of [1, 0, 0, 0, 0, 0, 0, 0, 0, 0] for a T-shirt, it becomes [0.91, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01]. This reduces the incentive for the model to become overly confident and can improve generalisation. I used a smoothing factor of 0.1.

Cosine Annealing with Warmup

To make the training process smoother, I used a cosine annealing learning-rate schedule with a short warm-up phase. The learning rate starts off very small and gradually increases for the first few epochs (I used three), until it reaches the maximum value of 0.001. After that, it slowly drops back down, following a cosine-shaped curve. The schedule was configured with a minimum learning rate of 0.000001, although the learning rate printed at the end of the 15-epoch training run was approximately 0.000018.

The formula looks like this:

$$lr = min_lr + (base_lr - min_lr) \times 0.5 \times (1 + \cos(\pi \times progress))$$

Starting with a small learning rate lets the model "find its footing" before it starts making bigger updates. The warm-up was intended to reduce the risk of unstable updates at the beginning of training. Then, as the cosine curve decays, the learning rate shrinks gradually, which helps fine-tune the model during the last few epochs. The resulting training curves showed a more consistent improvement in test accuracy during the later stages of training.

Implementation Details

For implementation, I used **Python and PyTorch 2.x**, with HuggingFace's Transformers library to handle ViT. Torchvision handled data loading and augmentation and scikit-learn was used for computing evaluation metrics. For plotting and analysis, I used Matplotlib and seaborn.

Hardware setup:

- **Laptop:** MacBook Pro (13-inch, 2017)
- **CPU:** Intel Core i5, 2.3 GHz (Turbo up to 3.6 GHz)
- **GPU:** Intel Iris Plus Graphics 640
- **RAM:** 8 GB

Despite the limited hardware, training went well because of the smaller model sizes and efficient augmentations.

- The **baseline model** took roughly 325.86 minutes (about 5.4 hours) to train.
- The **modified model** trained in only 142.99 minutes (about 2.4 hours).

Resulting in a 56.1% reduction in recorded training time. However, because the architecture, augmentation, loss function, learning-rate schedule, and number of training epochs were changed between the two experiments, the individual contribution of each change to the reduction in training time cannot be determined from the original experiment.

Experiments and Results

Dataset

The dataset used throughout all experiments was the **Fashion-MNIST** benchmark. It includes 60,000 training and 10,000 test grayscale images, each 28x28 pixels, across 10 fashion categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. It's balanced, with exactly 6,000 training images per class.

Before starting the training process, I resized all the Fashion-MNIST images from 28x28 to 32x32 pixels so that they would line up properly with the ViT's patch size. I also normalized the pixel values, scaling them from the original [0, 255] range to between -1 and 1. This provided a consistent input range for the models during training.

Even though the dataset itself is quite small and simple, it still turned out to be a good test of how well a model can tell apart similar types of clothing - for instance, "shirt" and "t-shirt." They look nearly identical in grayscale, which really pushes the model to learn subtle differences instead of just memorizing textures or outlines.

Evaluation Metrics

Several classification metrics were used to keep the evaluation fair and detailed :

- **Accuracy:** how many predictions were correct overall.
- **Macro F1-score:** the average F1-score across all classes, treating each one equally.
- **Weighted F1-score** – similar to above but gives more weight to classes with more samples.
- **Precision and Recall:** how well the model avoids false positives and false negatives.
- **Confusion Matrix:** a visual layout showing which classes were predicted correctly or confused.

Baseline Results

For the plain Vision Transformer, the results were:

- **Test Accuracy:** 83.80%
- **Training Time:** around 325.9 minutes
- **Final Training Loss:** 0.4505
- **Final Test Loss:** 0.4421

It was a strong baseline to start from: the training loss generally decreased over the 10 epochs, although test accuracy and test loss fluctuated between epochs. The final epoch achieved the highest test accuracy of 83.80%.

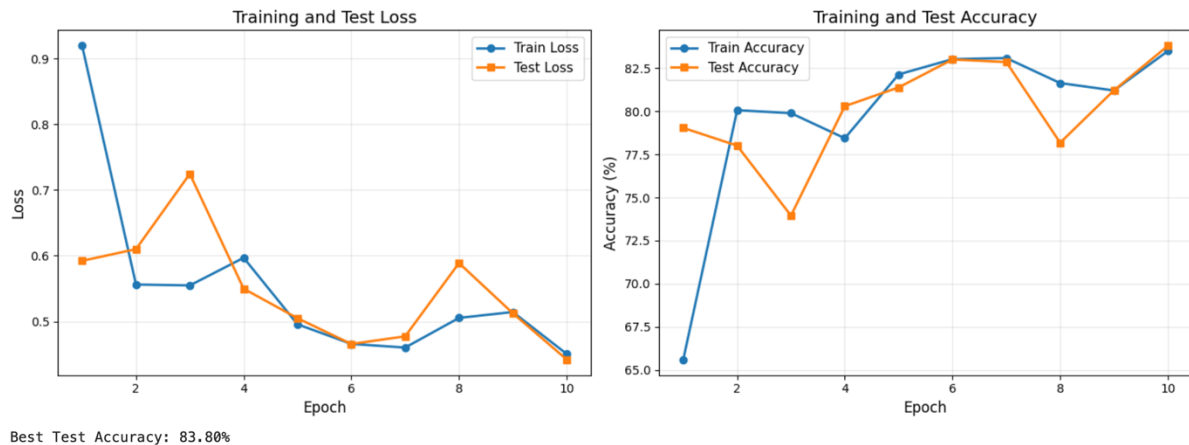


Figure 1: baseline training curves

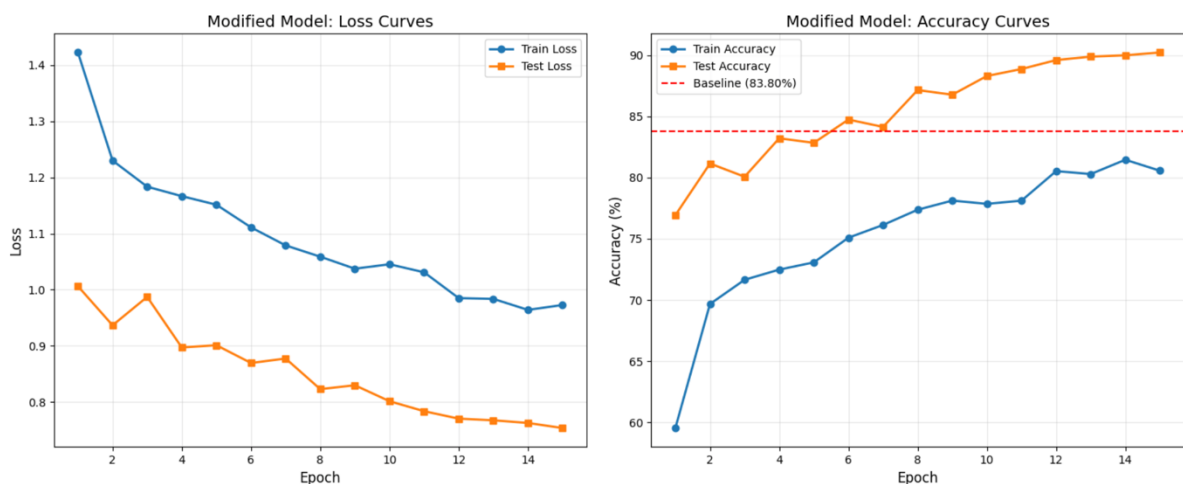
Modified Model Results

Once I introduced all three changes: the CNN front-end, stronger augmentation and better training schedule, the model's results improved quite a bit:

- **Test Accuracy:** 90.22%
- **Training Time:** 142.99 minutes
- **Macro F1-score:** 0.9015
- **Macro Precision:** 0.9018
- **Macro Recall:** 0.9022

The final model achieved a **6.42 percentage-point improvement in test accuracy** compared with the baseline, increasing from 83.80% to 90.22%. It also reduced the recorded training time by **56.1%**, from 325.86 minutes to 142.99 minutes.

The learning curves showed a more consistent improvement in test accuracy during the later stages of training, while the baseline exhibited greater variation between epochs. These results demonstrate that the combined architectural and training modifications were effective in improving the overall performance of the model. However, because all three modification groups were introduced together, the individual contribution of each change cannot be determined from this experiment alone.



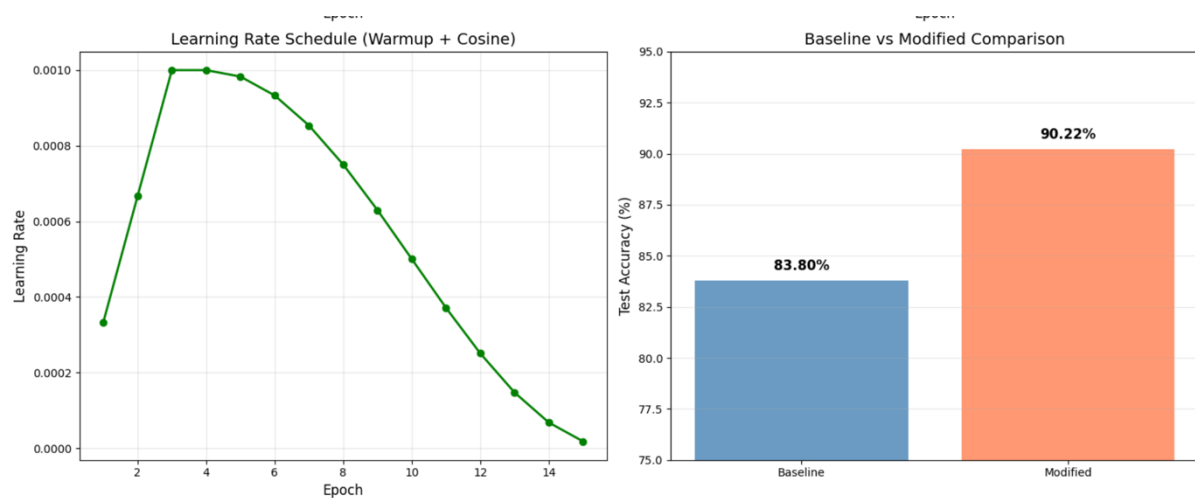


Figure 2: modified model results

Confusion Matrix Analysis

From the confusion matrix for the modified model, the improvements were clear. Classes that had looked confusing to the baseline model- like “shirt” and “t-shirt”- were much better separated now. The model also got more confident about items that had very diverse shapes, such as shoes, bags, and trousers.

The few mistakes that did remain made sense when you actually looked at the images. Some shirts in the dataset really do look like t-shirts, and coats sometimes resemble pullovers because they’re all grayscale and cropped tightly. The errors were therefore concentrated in visually similar categories rather than being evenly distributed across all classes.

Overall, the confusion matrix showed more diagonal dominance, meaning the predictions were correctly clustered along the main diagonal (true labels matching predicted ones). That’s a good sign that the classifier was learning stable, meaningful distinctions across categories.

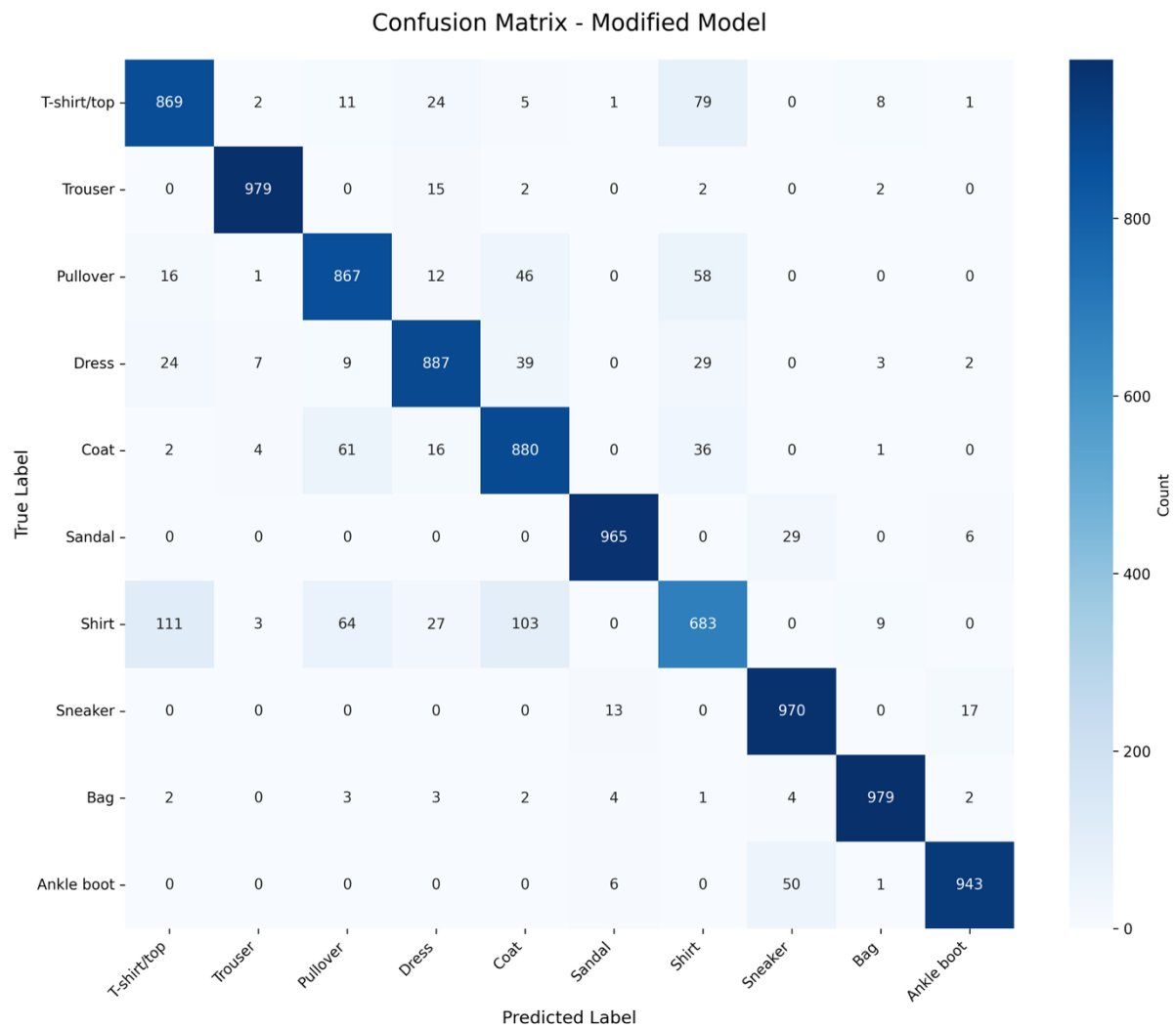


Figure 3: confusion matrix modified

Per-Class Performance

Breaking down performance by category, I noticed some clear patterns.

Best-performing classes: items like *bags*, *sneakers*, and *trousers* all had F1-scores above 0.92. These are visually distinct and easy for the model to recognize, even in grayscale.

Mid-range performers: categories like *coats* and *dresses* scored somewhere between 0.86 and 0.90. The model performed well with them, but there was occasional confusion between visually similar categories like shirts and coats.

Harder classes: the lowest score was for *shirts*, with an F1-score of 0.7235. This was followed by *coats* (0.8474) and *t-shirts* (0.8587). These categories have visually similar silhouettes, making them more difficult to distinguish in the grayscale images.

Overall, the hybrid CNN–Transformer achieved strong performance across the ten categories, with particularly high scores for visually distinctive items. The CNN front-end was designed to extract local visual features, while the transformer component could model relationships between these features across the image. However,

the experiment did not isolate the contribution of each modification, so these results should be interpreted as the effect of the combined hybrid approach rather than evidence that one component alone caused the improvement.

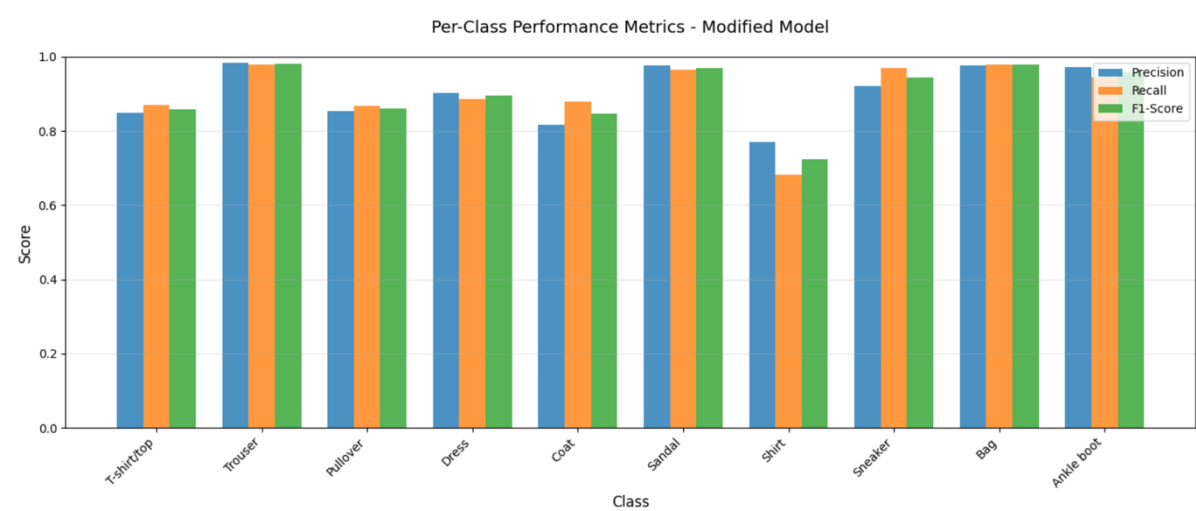


Figure 4: per class performance metrics

Baseline vs. Modified Model Comparison

Here’s a summary comparing both models side by side:

Metric	Baseline	Modified Model	Change
Test Accuracy	83.80%	90.22%	+6.42 percentage points
Training Time	325.86 min	142.99 min	−56.1%
Parameters	3.19M	3.45M	+8.2%
Macro F1	n/a	0.9015	n/a

The modified model had around 8% more parameters but still recorded a substantially lower training time and achieved higher test accuracy. However, because several changes were introduced together, including the CNN feature extractor, data augmentation, MixUp, label smoothing, and learning-rate scheduling, the experiment cannot determine which individual modification was responsible for the improvement.

Even though the parameter count increased slightly, the overall trade-off was favourable: the modified model achieved a 6.42 percentage-point improvement in test accuracy while using only around 8% more parameters. The cosine learning-rate schedule with warm-up was also part of the modified training strategy and may have contributed to the training behaviour observed, although its individual effect was not isolated in this experiment.

Discussion

When I looked at all three modifications together, it seemed that they may have complemented each other rather than working independently. The CNN layers gave the transformer a different starting representation by extracting local visual features such as edges and shapes before the data was passed to the ViT. This may have helped the model learn useful visual representations earlier in training, although the individual effect of the CNN was not isolated in this experiment.

The data-augmentation techniques, especially MixUp, were intended to reduce over-reliance on individual training examples and encourage the network to learn more general patterns. Similarly, the improved training strategy was designed to make the optimisation process more controlled. Label smoothing reduced the tendency

towards overconfident predictions, while the cosine learning-rate schedule with warm-up provided a gradual increase followed by a controlled reduction in the learning rate.

Another interesting result was how much lower the recorded training time was for the modified model. The hybrid model trained in 142.99 minutes compared with 325.86 minutes for the baseline, despite having slightly more parameters and being trained for more epochs. However, I cannot attribute this reduction to the cosine schedule or CNN feature extraction alone because several changes were introduced at the same time. A future ablation study would be useful for testing which modifications contributed most to the improvement.

Limitations and Challenges

Even though the project went pretty smoothly overall, there were still a few limitations worth pointing out.

To start with, the dataset itself was quite basic. Fashion-MNIST is grayscale and fairly low-resolution, which makes it useful for testing out algorithms but not very close to real-world data. In real product photos, clothes come in full colour, different lighting conditions, and much higher detail, so a model that works well here might not perform the same way in a real retail setting.

Another limitation is that the hybrid model, while achieving higher accuracy and a lower recorded training time, ended up having about 8.2% more parameters than the baseline. This is not a huge increase, but when scaling to larger datasets or deploying on smaller devices, additional parameters could affect memory usage and computational requirements.

I also didn't have enough compute power or time to do a deep hyperparameter search. Things like adjusting the patch size, experimenting with different cosine schedules, or tweaking the CNN depth could potentially improve the results further. As I was running everything on limited hardware, I couldn't explore more advanced models such as Swin Transformers or deeper CNN backbones either, which could have been interesting to try.

Another important limitation is that I introduced several modifications at the same time. This means the experiment shows the overall effect of the combined approach, but it does not show which individual modification contributed most to the improvement. A proper ablation study would be needed to investigate this.

Even with these constraints, the project achieved its main goal: improving accuracy and reducing recorded training time in a measurable way.

Comparison to Published Work

The paper that inspired my project was by Abd Alaziz et al. (2023), and they reported about 95.25% accuracy on Fashion-MNIST using a Vision Transformer-based approach. My best result was 90.22%, so it was lower than their reported result. However, the two experiments used different model configurations and training setups, so the results are not directly comparable.

My main aim wasn't really to beat their results. I wanted to see how far I could push performance by adjusting the architecture and training setup using the hardware I already had. The project showed me that meaningful improvements can be achieved without simply

increasing model size or using a much larger dataset. Combining CNN-based feature extraction with a transformer was one approach that produced a noticeable improvement in this experiment.

What I'd Do Next

- **Ablation study:** test the CNN, augmentation/MixUp, and training-strategy changes separately to determine which modifications contribute most to the final performance.
- **Model design:** experiment with deeper CNN layers or larger transformer stacks to see how far the hybrid approach can be improved. It would also be interesting to test architectures such as Swin Transformer, which use hierarchical attention.
- **Hyperparameter tuning:** systematically test factors such as patch size, CNN depth, learning rates, and training schedules.
- **Visualisation:** visualize the attention maps to investigate which parts of the images the model focuses on. This could make the model more interpretable and easier to explain.

These ideas would build on what I've done already and could make the approach more useful beyond this controlled dataset.

Conclusion

Looking back, I'd say the project turned out to be a success. I started with a basic Vision Transformer that scored 83.80% on Fashion-MNIST and managed to improve this to 90.22% after introducing several modifications. On top of that, the recorded training time dropped from 325.86 minutes to 142.99 minutes, a reduction of 56.1%.

What stood out most to me during this project was that there isn't necessarily a single trick to improving performance. It is more about finding the right combination of architectural and training choices. The CNN layers were intended to capture local visual features, while the transformer was used to model relationships between features across the image. The data augmentation and MixUp were intended to encourage better generalisation, while label smoothing and the learning-rate schedule were used to improve the training process.

Because these changes were introduced together, I cannot say that any one component was solely responsible for the improvement. However, the combined approach produced a substantial increase in test accuracy while using only around 8.2% more parameters than the baseline.

If there's one takeaway, it's that useful progress can be made without extremely large datasets or high-end computing resources. Understanding how different components fit together, testing them carefully, and analysing the results can make a significant difference. That's what made this project both challenging and rewarding to complete.

References

- [1] H. M. Abd Alaziz et al., "Enhancing Fashion Classification with Vision Transformer (ViT) and Developing Recommendation Fashion Systems Using DINOVA2," *Electronics*, vol. 12, no. 20, p. 4263, Oct. 2023.

- [2] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in Proc. Int. Conf. Learning Representations (ICLR), 2021.
- [3] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond Empirical Risk Minimization," in Proc. Int. Conf. Learning Representations (ICLR), 2018.
- [4] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pp. 2818-2826, 2016.
- [5] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv preprint arXiv:1708.07747, 2017.
- [6] A. Vaswani et al., "Attention is All You Need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998-6008, 2017.

Appendix A: Code Documentation

The main modifications in my code are clearly marked with comments:

- `# MODIFICATION 1: for the hybrid CNN-ViT architecture`
- `# MODIFICATION 2: for data augmentation`
- `# MODIFICATION 3: for training improvements`

This makes it straightforward to trace what was changed from the baseline.

Appendix B: Reproducibility

To ensure reproducibility, I fixed all random seeds:

- `torch.manual_seed(42)`
- `numpy.random.seed(42)`
- `random.seed(42)`

The project repository includes:

fashion-mnist-cnn-vit

- `README.md`
- `requirements.txt`
- `.gitignore`
- `notebooks/`
 - `dataset_exploration.ipynb`
 - `baseline_vit.ipynb`
 - `modified_vit.ipynb`
 - `evaluation.ipynb`
- `results/`
 - `baseline_training_curves.png`
 - `modified_model_results.png`
 - `confusion_matrix_modified.png`
 - `per_class_metrics_modified.png`

- docs/
 - o report.pdf
- sources/
 - o sources.md

Requirements:

torch $\geq$ 2.0.0

torchvision $\geq$ 0.15.0

transformers $\geq$ 4.30.0

scikit-learn $\geq$ 1.3.0

matplotlib $\geq$ 3.7.0

seaborn $\geq$ 0.12.0

numpy $\geq$ 1.24.0

tqdm $\geq$ 4.65.0

pillow $\geq$ 9.5.0